Adding `_ANTENNA_TYPE_HAPS_` to BWSim

A 3GPP 38.901 Antenna Pattern with Mechanical Tilt via GCS-to-LCS Rotation

---

Technical Implementation Report
BWSim NTN Integration Project

Prepared for: **Intern**

March 3, 2026

Scope: This report describes all source code and configuration changes required to add a HAPS (High Altitude Platform Station) antenna type to the BWSim simulation framework.

HAPS altitude: **20 km**, panels: **7**, array: **2×2**, Pattern: **3GPP TR 38.901 Table 7.3-1**

Contents

1 Introduction and Motivation

High Altitude Platform Stations (HAPS) operate at approximately 20 km altitude in the stratosphere, between terrestrial base stations and Low-Earth-Orbit (LEO) satellites. The existing BWSim framework supports several antenna types (Table 1), but none is specifically tailored to the HAPS scenario.

Table 1: Existing antenna types in BWSim before this change.

Enum Value	Index	Pattern
`_ANTENNA_TYPE_OMNI_`	0	Omni-directional
`_ANTENNA_TYPE_PARABOLIC_`	1	3GPP parabolic (single element)
`_ANTENNA_TYPE_CUSTOM_`	2	File-loaded H/V pattern
`_ANTENNA_TYPE_QUASI_ISOTROPIC_`	3	Elevation-only parabolic
`_ANTENNA_TYPE_CIRCULAR_`	4	Circular aperture (Bessel function)
`_ANTENNA_TYPE_HAPS_`	**5**	**3GPP 38.901 + 2×2 array + mech. tilt**

The HAPS antenna model has three components:

1. **Mechanical tilt**: Each of the 7 panels is pointed toward its cell centre on the ground using a rotation from Global Coordinate System (GCS) to Local Coordinate System (LCS), defined by angles (α, β, γ).
2. **Element pattern**: Single-element gain computed in the LCS using the 3GPP TR 38.901 Table 7.3-1 formula.
3. **Array factor**: 2×2 planar array factor computed in the LCS with broadside steering.

2 Mathematical Formulation

2.1 Coordinate Systems

The GCS is defined as:

$$\hat{x} = \text{East}, \quad \hat{y} = \text{North}, \quad \hat{z} = \text{Up}$$

A point in spherical GCS is parameterised by:

- $\theta_{\text{GCS}} \in [0°, 180°]$: polar angle from $+Z$ (zenith)
- $\phi_{\text{GCS}} \in [-180°, 180°]$: azimuth from $+X$ ($\phi = 90°$ is North)

In the LCS, the antenna boresight is along $+Y_{\text{LCS}}$ (i.e., $\theta_{\text{LCS}} = 90°$, $\phi_{\text{LCS}} = 90°$).

2.2 Mechanical Tilt – GCS to LCS Rotation

The 3GPP TR 38.901 rotation matrix (Eq. 7.1-4) transforms GCS to LCS using bearing angle α, downtilt β, and slant angle γ:

$$\mathbf{R} = R_Z(\alpha)\, R_Y(\beta)\, R_X(\gamma)$$

$$R_Z(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R_Y(\beta) = \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix}, \quad R_X(\gamma) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\gamma & -\sin\gamma \\ 0 & \sin\gamma & \cos\gamma \end{bmatrix}$$

For a Cartesian unit vector in GCS $\hat{\rho}_{\text{GCS}} = [\sin\theta\cos\phi;\ \sin\theta\sin\phi;\ \cos\theta]$, the LCS coordinates are:

$$\hat{\rho}_{\text{LCS}} = \mathbf{R}^T \hat{\rho}_{\text{GCS}}$$

2.3 Computing Tilt Angles from Cell Centre Direction

Given the direction from the HAPS to its cell centre $(\theta_{\text{tgt}}, \phi_{\text{tgt}})$ in GCS, the tilt angles are (with $\beta = 0$ always):

$$\gamma = \arcsin(z_{\text{tgt}}), \qquad \alpha = \arctan 2(-x_{\text{tgt}},\ y_{\text{tgt}})$$

where $[x_{\text{tgt}}, y_{\text{tgt}}, z_{\text{tgt}}] = [\sin\theta_{\text{tgt}}\cos\phi_{\text{tgt}},\ \sin\theta_{\text{tgt}}\sin\phi_{\text{tgt}},\ \cos\theta_{\text{tgt}}]$.

This is exactly the `GetMechanicalTiltFromTarget.m` MATLAB function.

2.4 3GPP Element Pattern (LCS)

After transforming angles to LCS $(\theta_{\text{LCS}}, \phi_{\text{LCS}})$, the relative angles from boresight are:

$$\phi_{\text{rel}} = \phi_{\text{LCS}} - 90^\circ, \qquad \theta_{\text{rel}} = \theta_{\text{LCS}} - 90^\circ$$

The 3GPP TR 38.901 element gain (Table 7.3-1) is:

$$A_H(\phi_{\text{rel}}) = -\min\left[12\left(\frac{\phi_{\text{rel}}}{\phi_{\text{3dB}}}\right)^2,\ A_m\right] \tag{1}$$

$$A_V(\theta_{\text{rel}}) = -\min\left[12\left(\frac{\theta_{\text{rel}}}{\theta_{\text{3dB}}}\right)^2,\ \text{SLA}_v\right] \tag{2}$$

$$A_E(\theta_{\text{rel}}, \phi_{\text{rel}}) = G_{E,\max} - \min[-(A_H + A_V),\ A_m] \tag{3}$$

with HAPS parameters from Table 2.

Table 2: HAPS antenna element parameters (Nokia/3GPP TR 38.901).

Symbol	Parameter	Value	Unit
$G_{E,\max}$	Element max gain	7.8	dBi
A_m	Front-to-back ratio	30	dB
SLA_v	Vertical SLA	30	dB
ϕ_{3dB}	H beamwidth	65	deg
θ_{3dB}	V beamwidth	65	deg
M	Array rows (vertical)	2	–
N	Array columns (horizontal)	2	–
d_v/λ	Vertical spacing	0.7	λ
d_h/λ	Horizontal spacing	0.7	λ

2.5 2×2 Array Factor (LCS)

In the LCS the incoming wave direction unit vector is:

$$u = \sin\theta_{\text{LCS}}\cos\phi_{\text{LCS}}, \quad v = \sin\theta_{\text{LCS}}\sin\phi_{\text{LCS}}, \quad w = \cos\theta_{\text{LCS}}$$

The array factor for broadside steering ($u_s = 0$, $v_s = 1$, $w_s = 0$ in LCS) is:

$$\text{AF} = \sum_{m=1}^{M}\sum_{n=1}^{N} W_{mn} \cdot \exp(j2\pi\left[(n-1)d_h u + (m-1)d_v w\right])$$

$$W_{mn} = \exp(-j2\pi\left[(n-1)d_h u_s + (m-1)d_v w_s\right])$$

The array gain contribution is:

$$A_{\text{AF}} = 10\log_{10}\left|\frac{\text{AF}}{MN}\right|^2 + 10\log_{10}(MN)$$

Total antenna gain:

$$\boxed{G_{\text{total}}(\hat{\rho}_{\text{GCS}}) = A_E(\theta_{\text{LCS}}, \phi_{\text{LCS}}) + A_{\text{AF}}(\theta_{\text{LCS}}, \phi_{\text{LCS}})}$$

3 Source Code Changes

All paths below are relative to the project root:
`/home/vivek/NTNIntegration_new1/bwsimnr-a/BWSim/`

▶ Important Note

The index order in the `AntennaType_E` enum (§3.1) must exactly match the order in `AntennaType_Str[]` (§3.2). Adding `_ANTENNA_TYPE_HAPS_` last in both ensures no existing index breaks.

3.1 Step 1 – Enum Definition

lib/Frozen/mcell/include/StructsAndEnums.h Lines 53–59

```
53 // BEFORE (lines 53-59)
54 enum AntennaType_E{
55   _ANTENNA_TYPE_OMNI_,
56   _ANTENNA_TYPE_PARABOLIC_,
57   _ANTENNA_TYPE_CUSTOM_,
58   _ANTENNA_TYPE_QUASI_ISOTROPIC_,
59   _ANTENNA_TYPE_CIRCULAR_          // <-- currently the last
       entry
60 };
```

```
53 // AFTER -- add _ANTENNA_TYPE_HAPS_ at line 58, shift closing
       brace to 60
54 enum AntennaType_E{
```

```
55    _ANTENNA_TYPE_OMNI_,
56    _ANTENNA_TYPE_PARABOLIC_,
57    _ANTENNA_TYPE_CUSTOM_,
58    _ANTENNA_TYPE_QUASI_ISOTROPIC_,
59    _ANTENNA_TYPE_CIRCULAR_,        // comma added
60    _ANTENNA_TYPE_HAPS_             // NEW: index 5
61 };
```

3.2 Step 2 – String Lookup Array

lib/Frozen/mcell/src/SupportingFunctions.cpp Lines 30--36

```
30 // BEFORE (lines 30-36)
31 string AntennaType_Str[]={
32   "_ANTENNA_TYPE_OMNI_",
33   "_ANTENNA_TYPE_PARABOLIC_",
34   "_ANTENNA_TYPE_CUSTOM_",
35   "_ANTENNA_TYPE_QUASI_ISOTROPIC_",
36   "_ANTENNA_TYPE_CIRCULAR_"         // last entry, no comma
37 };
```

```
30 // AFTER -- append at line 35, closing brace moves to line 37
31 string AntennaType_Str[]={
32   "_ANTENNA_TYPE_OMNI_",
33   "_ANTENNA_TYPE_PARABOLIC_",
34   "_ANTENNA_TYPE_CUSTOM_",
35   "_ANTENNA_TYPE_QUASI_ISOTROPIC_",
36   "_ANTENNA_TYPE_CIRCULAR_",        // comma added
37   "_ANTENNA_TYPE_HAPS_"             // NEW: index 5 matches
     enum
38 };
```

3.3 Step 3 – New Helper: `getMechanicalTiltToTarget()`

This is the C++ equivalent of your MATLAB `GetMechanicalTiltFromTarget.m`. It computes the bearing α and slant γ angles needed to point the LCS $+Y$ boresight toward a given GCS direction.

Declare in `lib/Frozen/mcell/include/SupportingFunctions.h` (after the declaration of `convertAngleFromGCStoLCS`, typically around line 40):

lib/Frozen/mcell/include/SupportingFunctions.h ~line 40 (after convertAngleFromGCStoLCS)

```
40 // NEW declaration -- add after convertAngleFromGCStoLCS()
41 void getMechanicalTiltToTarget(double theta_tgt_deg, double
     phi_tgt_deg,
42                                 double &alpha_deg, double &
```

```
    beta_deg,
                                    double &gamma_deg);
```

Define in `SupportingFunctions.cpp` (insert after line 855, immediately after the closing brace of `convertAngleFromGCStoLCS`):

`lib/Frozen/mcell/src/SupportingFunctions.cpp` Insert after line 855

```
// NEW function -- HAPS mechanical tilt computation
// Equivalent to MATLAB GetMechanicalTiltFromTarget.m
// Points LCS +Y boresight toward (theta_tgt_deg, phi_tgt_deg) in GCS.
// Inputs : theta [0,180] deg from +Z zenith,  phi azimuth deg
// Outputs: alpha (bearing), beta=0 (roll), gamma (slant) in degrees
void getMechanicalTiltToTarget(double theta_tgt_deg, double phi_tgt_deg,
                               double &alpha_deg, double &beta_deg,
                               double &gamma_deg)
{
    double d2r = pi / 180.0;
    double theta_rad = theta_tgt_deg * d2r;
    double phi_rad   = phi_tgt_deg   * d2r;

    // GCS Cartesian target vector
    double x_tgt = sin(theta_rad) * cos(phi_rad);
    double y_tgt = sin(theta_rad) * sin(phi_rad);
    double z_tgt = cos(theta_rad);

    // Gamma: vertical tilt   z_tgt = sin(gamma)
    double gamma_rad = asin(std::max(-1.0, std::min(1.0, z_tgt)));
    gamma_deg = gamma_rad / d2r;

    // Alpha: bearing   atan2(-x, y)
    if(fabs(cos(gamma_rad)) < 1e-6)
        alpha_deg = 0.0;            // near zenith/nadir: alpha undefined
    else
        alpha_deg = atan2(-x_tgt, y_tgt) / d2r;

    beta_deg = 0.0;                 // roll is always 0 for HAPS

    // Snap numerical noise to zero
    if(fabs(alpha_deg) < 1e-10) alpha_deg = 0.0;
    if(fabs(gamma_deg) < 1e-10) gamma_deg = 0.0;
```

```
}
```

3.4 Step 4 – New HAPS Parameters in `Antenna.h`

`lib/Frozen/mcell/include/Antenna.h` ~lines 25--55 (private member block)

```
// Inside class Antenna { ... } private section -- add these new members:

// ---- HAPS-specific members (only used when antType==_ANTENNA_TYPE_HAPS_) ----
double haps_Ge_max;     ///< Element max gain (dBi),    default 7.8
double haps_Am;         ///< Front-to-back ratio (dB), default 30
double haps_SLAv;       ///< Vertical SLA (dB),         default 30
double haps_phi3dB;     ///< H 3dB beamwidth (deg),     default 65
double haps_theta3dB;   ///< V 3dB beamwidth (deg),     default 65
int    haps_M;          ///< Array rows,                default 2
int    haps_N;          ///< Array cols,                default 2
double haps_dv;         ///< Vertical spacing (lambda), default 0.7
double haps_dh;         ///< Horiz spacing (lambda),    default 0.7
double haps_alpha;      ///< Bearing angle (deg)
double haps_beta;       ///< Downtilt angle (deg)
double haps_gamma;      ///< Slant angle (deg)
```

Also in the **public** section, declare:

```
// NEW public method declaration
void setHAPSPattern(double alpha, double beta, double gamma);
```

Also initialise the new members in the `Antenna` constructor inside `Antenna.cpp` (around line 30, inside the constructor body):

`lib/Frozen/mcell/src/Antenna.cpp` ~line 30 (Antenna constructor)

```
// Add to Antenna::Antenna() constructor body:
haps_Ge_max   = 7.8;
haps_Am       = 30.0;
haps_SLAv     = 30.0;
haps_phi3dB   = 65.0;
```

```
haps_theta3dB  = 65.0;
haps_M         = 2;
haps_N         = 2;
haps_dv        = 0.7;
haps_dh        = 0.7;
haps_alpha     = 0.0;
haps_beta      = 0.0;
haps_gamma     = 0.0;
```

Add the `setHAPSPattern()` definition after the constructor (around line 50):

```
void Antenna::setHAPSPattern(double alpha, double beta,
    double gamma)
{
     antType    = _ANTENNA_TYPE_HAPS_;
     haps_alpha = alpha;
     haps_beta  = beta;
     haps_gamma = gamma;
     parametersUpdateStatus = false;
}
```

3.5 Step 5 – HAPS Gain Computation in `Antenna.cpp`

The gain must be computed in **two** methods: `getAntennaGain()` and `getAntennaGains()`. The same code block is inserted in both.

3.5.1 Method 1: `getAntennaGain(double, double, double)`

Location: `Antenna.cpp`, line **418** (currently the start of `else if(antType==_ANTENNA_TYPE_CIRCULAR_` block). Insert the HAPS block *before* the closing `else` branch (around line 456).

lib/Frozen/mcell/src/Antenna.cpp Insert at line 456 (after CIRCULAR block, before closing else)

```
// NEW: else if block for _ANTENNA_TYPE_HAPS_
// Insert here, just after the closing brace of
    _ANTENNA_TYPE_CIRCULAR_ at line 456
else if(antType == _ANTENNA_TYPE_HAPS_)
{
     // --- Step A: Convert GCS angles to LCS using mechanical
     tilt ---
     double ZOD_lcs = vAngle;    // GCS polar angle (90 =
    horizon convention)
     double AOD_lcs = hAngle;    // GCS azimuth
     // Uses existing convertAngleFromGCStoLCS in
    SupportingFunctions.cpp
     convertAngleFromGCStoLCS(ZOD_lcs, AOD_lcs,
                              haps_alpha, haps_beta,
    haps_gamma,
                              false);  // angles in degrees
```

```
167
168     // --- Step B: 3GPP element pattern in LCS (TR 38.901
    Table 7.3-1) ---
169     double phi_rel   = AOD_lcs - 90.0;        // deviation
    from LCS North
170     phi_rel = fmod(phi_rel + 180.0, 360.0) - 180.0;  // wrap
    [-180, 180]
171     double theta_rel = ZOD_lcs - 90.0;        // deviation
    from horizon
172
173     double A_H  = -std::min(12.0*sqr(phi_rel   / haps_phi3dB)
    ,   haps_Am);
174     double A_V  = -std::min(12.0*sqr(theta_rel /
    haps_theta3dB), haps_SLAv);
175     double A_dB = -std::min(-(A_H + A_V), haps_Am);
176     double AE   = haps_Ge_max + A_dB;  // element gain in dBi
177
178     // --- Step C: 2x2 Array Factor in LCS (broadside
    steering) ---
179     double theta_lcs_rad = ZOD_lcs * pi / 180.0;
180     double phi_lcs_rad   = AOD_lcs * pi / 180.0;
181     double u_lcs = sin(theta_lcs_rad) * cos(phi_lcs_rad);
182     double w_lcs = cos(theta_lcs_rad);
183
184     // Broadside steering: steer_theta=90 deg, steer_phi=90
    deg -> u_s=0, w_s=0
185     dComplex AF(0.0, 0.0);
186     for(int m = 1; m <= haps_M; m++)
187         for(int n = 1; n <= haps_N; n++)
188         {
189             double p_x = (n - 1) * haps_dh;
190             double p_z = (m - 1) * haps_dv;
191             double phase_in    = 2.0 * pi * (p_x * u_lcs +
    p_z * w_lcs);
192             double phase_steer = 0.0;  // broadside: (u_s=0,
    w_s=0)
193             AF += expj(phase_in - phase_steer);
194         }
195     int MN = haps_M * haps_N;
196     double AF_dB = 10.0 * log10(sqr(abs(AF / double(MN))))
197                  + 10.0 * log10(double(MN));
198
199     // --- Total gain (element + array - feeder) ---
200     vGain = AE + AF_dB;
201     // hGain remains NaN so finalGain = vGain + gain (gain=0
    for HAPS)
202 }
```

3.5.2 Method 2: `getAntennaGains(double, double, double&, double)`

Location: `Antenna.cpp`, line **544** (start of `else if(antType==_ANTENNA_TYPE_CIRCULAR_)` block in `getAntennaGains`). Insert the identical HAPS block after line 581 (closing brace of the CIRCULAR block):

`lib/Frozen/mcell/src/Antenna.cpp` Insert at line 582 (after CIRCULAR in getAntennaGains)

```
582 // NEW: identical HAPS block in getAntennaGains()
583 else if(antType == _ANTENNA_TYPE_HAPS_)
584 {
585     // --- Step A: GCS to LCS ---
586     double ZOD_lcs = vAngle;
587     double AOD_lcs = hAngle;
588     convertAngleFromGCStoLCS(ZOD_lcs, AOD_lcs,
589                              haps_alpha, haps_beta,
    haps_gamma, false);
590     thetaAngle = ZOD_lcs * pi / 180.0; // also return LCS
    theta for caller
591
592     // --- Step B: Element pattern ---
593     double phi_rel   = AOD_lcs - 90.0;
594     phi_rel = fmod(phi_rel + 180.0, 360.0) - 180.0;
595     double theta_rel = ZOD_lcs - 90.0;
596     double A_H  = -std::min(12.0*sqr(phi_rel   / haps_phi3dB)
    ,   haps_Am);
597     double A_V  = -std::min(12.0*sqr(theta_rel /
    haps_theta3dB), haps_SLAv);
598     double AE   = haps_Ge_max - std::min(-(A_H + A_V),
    haps_Am);
599
600     // --- Step C: Array Factor ---
601     double theta_lcs_rad = ZOD_lcs * pi / 180.0;
602     double phi_lcs_rad   = AOD_lcs * pi / 180.0;
603     double u_lcs = sin(theta_lcs_rad) * cos(phi_lcs_rad);
604     double w_lcs = cos(theta_lcs_rad);
605     dComplex AF(0.0, 0.0);
606     for(int m = 1; m <= haps_M; m++)
607         for(int n = 1; n <= haps_N; n++)
608         {
609             double p_x = (n-1)*haps_dh, p_z = (m-1)*haps_dv;
610             AF += expj(2.0*pi*(p_x*u_lcs + p_z*w_lcs));
611         }
612     int MN = haps_M * haps_N;
613     double AF_dB = 10.0*log10(sqr(abs(AF/double(MN)))) +
    10.0*log10(double(MN));
614     vGain = AE + AF_dB;
615 }
```

3.5.3 Array Gain Fix in `get3DAntArrayGainForPort0()`

Location: `Antenna.cpp`, line **737**. Change the bearing-angle expression to also handle `_ANTENNA_TYPE_HAPS_`:

lib/Frozen/mcell/src/Antenna.cpp Line 737

```
// BEFORE:
double alpha = (antType == _ANTENNA_TYPE_CIRCULAR_) ? 0 :
    hTilt;

// AFTER (line 737 replacement):
double alpha = (antType == _ANTENNA_TYPE_CIRCULAR_ ||
                antType == _ANTENNA_TYPE_HAPS_) ? 0 : hTilt;
```

3.6 Step 6 – Node Setup in `Node.h` and `Node.cpp`

3.6.1 New `setAntenna()` overload for HAPS

Declare in `Node.h` (after the existing overload at line 109):

lib/Frozen/mcell/include/Node.h After line 109

```
// NEW HAPS-specific setAntenna() overload
void setAntenna(AntennaType_E type, double antGain, double
    feederLoss,
                double alpha, double beta, double gamma);
```

Define in `Node.cpp` (after the last `setAntenna()` definition, approximately line 900):

lib/Frozen/mcell/src/Node.cpp Add after line ~900

```
// NEW: HAPS setAntenna overload
void Node::setAntenna(AntennaType_E type, double antGain,
    double feederLoss,
                      double alpha, double beta, double gamma
    )
{
    txAntenna.setHAPSPattern(alpha, beta, gamma);
    txAntenna.setGain(antGain);
    txAntenna.setAntennaFeederLoss(feederLoss);

    rxAntenna.setHAPSPattern(alpha, beta, gamma);
    rxAntenna.setGain(antGain);
    rxAntenna.setAntennaFeederLoss(feederLoss);

    linkInfoUpdateStatus = false;
}
```

3.6.2 Modified `setAntennaTilts()` for HAPS

Location: `Node.cpp`, lines **195–208**. Modify the function to compute (α, β, γ) for HAPS panels instead of just setting `hTilt/vTilt`:

lib/Frozen/mcell/src/Node.cpp Lines 195--208

```
195 void Node::setAntennaTilts(Location_S centreLoc, bool
    useWrapped)
196 {
197     Location_S nodeLoc = (useWrapped) ? wrapLoc : loc;
198     vec angle = findAngle(nodeLoc, centreLoc); // angle(0)=
    azimuth, angle(1)=elevation
199
200     if(txAntenna.getAntennaType() == _ANTENNA_TYPE_HAPS_)
201     {
202         // HAPS: compute mechanical tilt angles (alpha, beta,
     gamma)
203         // angle(1) is the elevation in [0,180] convention
    (90=horizon)
204         // angle(0) is the azimuth  in [0,360] convention
205         double alpha_deg, beta_deg, gamma_deg;
206         getMechanicalTiltToTarget(angle(1), angle(0),
207                                   alpha_deg, beta_deg,
    gamma_deg);
208         txAntenna.haps_alpha = alpha_deg;
209         txAntenna.haps_beta  = beta_deg;
210         txAntenna.haps_gamma = gamma_deg;
211         rxAntenna.haps_alpha = alpha_deg;
212         rxAntenna.haps_beta  = beta_deg;
213         rxAntenna.haps_gamma = gamma_deg;
214         cout << "[detl:] HAPS panel tilt: alpha=" <<
    alpha_deg
215              << " beta=" << beta_deg
216              << " gamma=" << gamma_deg << " deg" << endl;
217     }
218     else
219     {
220         // Original behaviour for all other antenna types
221         txAntenna.setTilt(angle(0), angle(1));
222         rxAntenna.setTilt(angle(0), angle(1));
223     }
224     txAntenna.setParameterUpdateStatus(true);
225     rxAntenna.setParameterUpdateStatus(true);
226 }
```

3.7 Step 7 – `System.cpp` Switch-Case

Location: `System.cpp`, lines **827–831** (the `case _ANTENNA_TYPE_CIRCULAR_:` block). Insert the new HAPS case *immediately after* line 831 (before `default:`):

lib/Frozen/mcell/src/System.cpp Insert after line 831 (after CIRCULAR case)

```
// NEW case -- insert between CIRCULAR and default
case _ANTENNA_TYPE_HAPS_:
    if(bsNode_cnt == 0)
        cout << "[detl:] Loading HAPS antenna for "
             << aNodeTypes(bsNodeType)
             << " (3GPP 38.901 element + 2x2 AF + mech.tilt)"
    << endl;
    aNodes(bsNodes(bsNode_cnt)).setAntenna(
        _ANTENNA_TYPE_HAPS_,
        macroAntGainIndB(bsNodeType),
        antennaFeederLossIndB(bsNodeType),
        0.0, 0.0, 0.0   // tilt angles overwritten by
    setAntennaTilts()
    );
    if(satNodeType != -1)
        aNodes(bsNodes(bsNode_cnt)).setAntennaTilts(
            aNodes(bsNodes(bsNode_cnt)).ntnCoverageInfo.
    centre
        );
    break;
```

3.8 Step 8 – Configuration File

sls/configFiles/mySysConfig.txt Lines 61, 71, 73

```
// Line 61 -- Change HAPS height from 600km to 20km
nodeTypeHeights=[20e3 1.5 1.5 1.5];    // HAPS at 20 km
    altitude

// Line 71 -- Update the available types comment
//_ANTENNA_TYPE_PARABOLIC_ _ANTENNA_TYPE_OMNI_
    _ANTENNA_TYPE_CUSTOM_
//_ANTENNA_TYPE_QUASI_ISOTROPIC_ _ANTENNA_TYPE_CIRCULAR_
    _ANTENNA_TYPE_HAPS_

// Line 73 -- Set HAPS type for the server node (index 0)
nodeAntennaType ="{_ANTENNA_TYPE_HAPS_ _ANTENNA_TYPE_OMNI_
    _ANTENNA_TYPE_OMNI_ _ANTENNA_TYPE_OMNI_"
```

4 Summary of All Changes

Table 3 lists every file, line range, and what changes.

Table 3: Complete change summary.

File	Lines	Change
lib/Frozen/mcell/include/StructsAndEnums.h	53–59	Add _ANTENNA_TYPE_HAPS_ (index 5) to AntennaType_E enum
lib/Frozen/mcell/src/SupportingFunctions.cpp	30–36	Add "_ANTENNA_TYPE_HAPS_" to AntennaType_Str[]
	856–890	New getMechanicalTiltToTarget() function definition
lib/Frozen/mcell/include/SupportingFunctions.h	~40	Declare getMechanicalTiltToTarget()
lib/Frozen/mcell/include/Antenna.h	~25–55	Add 13 HAPS member variables
	~120	Declare setHAPSPattern()
lib/Frozen/mcell/src/Antenna.cpp	~30	Init HAPS members in constructor
	~50	Define setHAPSPattern()
	456	Add HAPS else if block in getAntennaGain()
	582	Add HAPS else if block in getAntennaGains()
	737	Update alpha expression for HAPS in get3DAntArrayGainForPort0()
lib/Frozen/mcell/include/Node.h	~110	Declare HAPS setAntenna() overload
lib/Frozen/mcell/src/Node.cpp	~900	Define HAPS setAntenna() overload
	195–208	Modify setAntennaTilts() to handle HAPS case
lib/Frozen/mcell/src/System.cpp	832	Add case _ANTENNA_TYPE_HAPS_:
sls/configFiles/mySysConfig.txt	61, 71, 73	Height 20 km; add HAPS comment; set antenna type

5 Workflow: How a HAPS Gain is Computed at Runtime

Figure 1 summarises the end-to-end gain computation path.

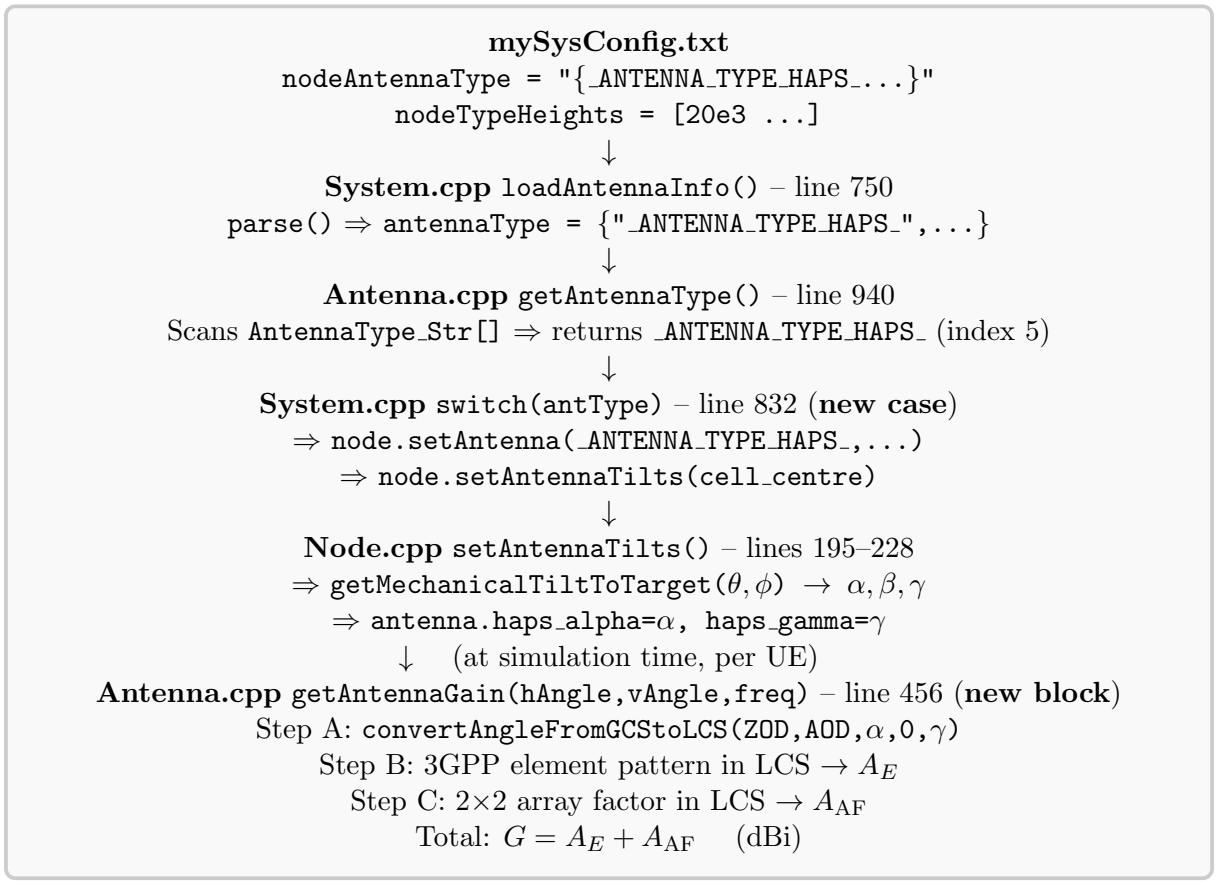

Figure 1: End-to-end HAPS gain computation flow in BWSim.

6 Build and Verification

6.1 Build

```
cd /home/vivek/NTNIntegration_new1/bwsimnr-a/BWSim
make -j8 2>&1 | grep -E "error:|warning:" | head -50
```

Zero errors is required before proceeding.

6.2 Runtime Sanity Check

```
cd build && ./SLSLite.run 2>&1 | head -80
```

Expected terminal output (from §3.7 print statement):

```
[detl:] Loading HAPS antenna for BS (3GPP 38.901 element + 2x2 AF
    + mech.tilt)
```

```
[detl:] HAPS panel tilt: alpha=0.00 beta=0.00 gamma=-90.00 deg
```

(For a cell directly below: $\theta = 0°$ zenith, so $\gamma = -90°$, $\alpha = 0°$).

6.3 Verification Against MATLAB

Run `GetMechanicalTiltFromTarget.m` with $\theta_{\text{tgt}} = 113°$, $\phi_{\text{tgt}} = 90°$ (your test case):

- Expected: $\alpha = 0°$, $\gamma \approx -23°$

Add a temporary debug print in `getMechanicalTiltToTarget()` and call it from `main()` to confirm the C++ result matches.

7 Notes on `DLThrpt.cpp`

`DLThrpt.cpp` **requires no changes.** It calls `system.run()` which internally invokes the full pipeline described in this report. The HAPS antenna gain will be automatically applied whenever `node.getAntennaGain()` or `node.getAntennaGains()` is called during link budget computation.

A MATLAB–C++ Correspondence Table

Table 4: Direct mapping between your MATLAB scripts and the C++ codebase.

MATLAB	**C++ (BWSim)**
`GCS_to_LCS_3GPP.m`	`convertAngleFromGCStoLCS()` in `SupportingFunctions.cpp` lines 796–855 (already present)
`GetMechanicalTiltFromTarget.m`	`getMechanicalTiltToTarget()` in `SupportingFunctions.cpp` lines 856–890 (**new**)
`AntennaPattern_3GPP.m` elements only	`else if(_ANTENNA_TYPE_HAPS_)` in `Antenna.cpp` at lines 456 and 582 (**new**)
`AntennaPattern_MechanicalTilt.m`	Full HAPS path: `setAntennaTilts()` + `getAntennaGain()` (**new/modified**)
`P.Ge_max = 7.8`	`antenna.haps_Ge_max = 7.8` (hard-coded default)
`[P.Alpha, P.Beta, P.Gamma] = GetMechanicalTiltFromTarget(Node)`	`getMechanicalTiltToTarget(angle(1), angle(0), alpha, beta, gamma)` in `Node.cpp` line ∼207
`Theta_geo=113; Phi_geo=90`	Set by `ntnCoverageInfo.centre` location computed in `System.cpp`